Jump-starting AI agents

A practical quick start for economists

Simon Martin

simon.martin@univie.ac.at
sites.google.com/view/simonmartinecon
github.com/simonmartinecon/jumpstart-ai

Version 0.1, October 2026

These slides focus on practical aspects. There is, of course, much discussion about the ethics of AI use (in writing and reviewing), data policies and more; assessing these is up to you.

About these slides

- **Objective:** get you started on one of your own projects right after skimming these slides.
- **For whom:** economists in academia, but much of this applies more widely.
- **Prerequisites:** almost none. To start on your project right away, you need a basic subscription (about 20 USD/month) with any AI provider.
- **Caveat:** this is my personal take in a field that changes very fast. It is what works for me, not a fixed recipe: adjust it to your own workflow and software.
- **Meta:** these slides were made with an agent, following exactly the workflow they describe. The full project folder is on GitHub (github.com/simonmartinecon/jumpstart-ai).

Happy to discuss with anyone, any time: simon.martin@univie.ac.at

Chatbot vs agent

	Chatbot	Agent
Input	you copy/paste text into the chat	reads your files directly
Output	you copy/paste answers back into your documents	writes into your files (text, code, tables)
Context	the current chat only	your whole project folder
Tasks	answers one question at a time	plans and carries out multi-step tasks on its own

The basics on these slides get you maybe 90–95% of the gains. Ignore advanced features (skills, GitHub integration, . . .) for now: start simple.

“Coding agent”: not just for coding

- **Misconception 1: coding is for programmers.** Almost everything you do on a computer is “coding”: writing and formatting text, reading documents, calculating, combining content from several files.
- **Misconception 2: it is for empirical work only.** Agents work just as well for writing, admin and theory; their maths skills have improved a lot recently.

Writing

Check that the introduction of paper.tex is consistent with Section 5.

Admin

Draft the progress report from the documents in funding/.

Theory

Turn my notes in notes.md into a formal proposition, in the style of the samples in samples/.

Empirics

Add clustered standard errors to all regressions in code/analysis.do.

What it is, and what it isn't

"Just a token-guessing machine?"

- Technically, yes: it predicts the next piece of text.
- But predicting well at this scale means it can follow arguments, plan tasks and check proofs.
- Judge it by its results on your own tasks.

Not a store of truth

- It is not a database, and not Wikipedia.
- It reconstructs facts rather than looking them up, so they can be wrong, outdated or invented (citations!).

Strong at working with what you give it, weak at recalling facts. So provide the sources (literature/, data/), and let it compute in code instead of answering from memory (see slides 13 and 17).

What you need

Getting started

- **Subscription:** any basic plan at about 20 USD/month (Claude, ChatGPT/Codex, Gemini, . . .). Free plans are usually too limited; 200 USD plans are rarely needed. The provider matters less than you might think.
- **Software:** an editor with the agent built in, e.g. VS Code with the Claude Code extension (my setup).
- **Setup:** ask the agent to install or explain LaTeX, Stata paths etc., and to convert existing .docx/.pdf drafts to .tex/.md once.

Settings

- **Model:** any current model is fine. No need to chase every new release.
- **Effort:** medium for most tasks; it trades depth against speed and tokens. High only for occasional thorough checks.
- **Tokens:** the 20 USD plan suffices for most daily use. What eats tokens: many parallel web searches, and binary files (.docx, .xlsx, .pdf) instead of plain text.

Your agent is your research assistant

- Extremely capable, but only as good as the context you give it.
- Your job, as with any new RA:
  - **Context:** background, related literature, your terminology. A brilliant RA with a physics background needs to know how you use certain terms.
  - **Instructions:** what to do, which output you expect, and in which file.
  - **Unclear points:** say how to handle them, for example:

💬 *Never make assumptions. Always clarify with me first.*

💬 *Proceed as far as you can. Leave a TODO note in the document for anything unclear, so we can come back to it later.*

- You do most of this once, in `instructions.md` (see slide 14).

You stay in charge

- RAs make mistakes and misread input, and so does your agent. **You remain responsible for the final output.**
- **Verify:** read and check everything, and leave enough time for it at the end. AI slop shows.
- A second agent can cross-check for you, but the responsibility stays with you.
- **Memory drifts:** as a session grows, older or partly contradictory instructions get lost. Don't assume everything you ever said is still respected; now and then ask it to re-read the instructions or double-check.
- **Fresh sessions:** start new sessions rather than one endless chat. Let the agent record progress and open points in `status.md`, and read it at the start of each session.

A powerful tool, not a substitute for you

Used wisely: a very powerful tool

- Used properly, it can do a lot of the work, and do it fast.
- It can also sharpen your own understanding: let it explain a method, a proof step or a piece of code, and let it challenge your arguments.

💬 *Walk me through what you changed in analysis.do, and why.*

Not: "it does it all for me"

- Don't make yourself redundant.
- Stay on top of what is happening: understand every step, table and argument, as you would with an RA's work.

💬 *Explain step by step why this estimator is consistent under our assumptions.*

💡 Rule of thumb: you should be able to explain and defend every part of the output yourself.

Project = folder

- Agents work in **projects**, and a project is simply a folder on your computer.
- Everything the agent needs goes in there, and you come back to the same folder each time. Many projects are short-lived; that is fine.

A research paper

my-paper/
- instructions.md . . . . . the contract
- status.md . . . . . progress log
- paper.tex . . . . . main document
- literature/ . . . . . PDFs you may cite
- samples/ . . . . . your writing samples
- data/ . . . . . raw data
- code/ . . . . . do files
- output/ . . . . generated tables, figures

These slides

jumpstart-ai/
- instructions.md . . . . . the contract
- status.md . . . . . progress log
- background.md . . . . . my notes
- recordings/ . . . . . voice transcripts
- examples/ . . . . . code shown on slides
- slides.tex . . . . . main document
- slides.pdf . . . . . final output
- README.md . generated from slides.tex

Plain text for everything you work on

Plain text	Binary
.tex, .md, .txt, .csv, .do, .R, .py	.docx, .xlsx, .pdf, .dta, .png
→ everything you actively work on	→ only pure input (literature, external files) and final output (the compiled PDF)

- **Why:** plain text is close to how the agent reads: much faster, far fewer tokens, and much easier to edit.
- **Example:** to find GDP for Austria in 2020, the agent first has to decode an .xlsx file. In a .csv, one quick search returns the number.
- **Bonus:** changes over time are easy to see (what changed, when, and why), even with simple Dropbox file history.

Try it: open a .docx or .pdf in Notepad or TextEdit. That is what the agent has to work through.

No copy/paste

- Copy/paste is friction: you should never need it.
- **Input:** point the agent to where things are.

> *Consider Smith2023.pdf in literature/.*

> *Rework paragraph 3 of the introduction in paper.tex.*

> *Check the equation in line 142 of paper.tex.*

- **Output:** have it write straight into the final file. It finds the right place.

> *Slide 17 is fine as discussed. Write it into slides.tex.*

- Never copy answers from the chat window into your documents.

Script everything

- Every table, figure and number in your document comes from a script. Never edit generated tables by hand: you will redo each one ten more times, so make it come out right from the code.
- You don't need to know how to script an operation: the agent does it for you.

code/wage_regs.do writes the table:

```
* Example data shipped with Stata
sysuse nlsw88, clear
generate lwage = ln(wage)

* Regressions
eststo clear
eststo: regress lwage grade ttl_exp, robust
eststo: regress lwage grade ttl_exp union, robust

* Export a production-ready LaTeX table
esttab using "output/tables/wage_regs.tex", ///
    replace booktabs label se               ///
    keep(grade ttl_exp union)               ///
    mtitles("Baseline" "With union")
```

paper.tex includes it:

```
\begin{table}
  \caption{Wage regressions}
  \input{output/tables/wage_regs.tex}
\end{table}
```

✗ Don't	✓ Do
Change the column order of Table 2 in paper.tex.	Change wage_regs.do so that Table 2 comes out in this column order.
What does this coefficient imply in percent? (Then paste the answer.)	Add this calculation to the do-file, export the number, and use it in Section 4.

instructions.md: the contract

Section	My typical content
Scope	*We are writing a research paper in labour economics.*
Literature	*Read the papers in literature/.*
Stages	sub-steps, and how we proceed and interact (longer projects)
Output	*The main document is paper.tex.*
Software	which software; where code, input and output files are
Style	a writing sample of mine in samples/: sets layout, language and tone
Checks	*At the end: grammar, language, style and consistency check.*
Logging	*Record progress and TODOs in status.md; read it at the start of each session.*

- Instead of long one-off prompts: one file that sets the scope and the rules.
- **Format barely matters:** no syntax rules, .md or .txt. A little structure (headings) helps.
- **Why bother:** 10–15 minutes of writing forces you to think through background, deliverables and sub-steps, and tells your RA clearly what to do.
- Extend it as you go.

Example: the `instructions.md` behind these slides

```
# Instructions

## Scope
We are writing an overview on jump-starting AI agents for
       economists. The main goal is to help other economists get
        started as quickly as possible: only the bare-bones
       essentials to keep in mind, with no unnecessary detail,
       background information or technical lingo. Readers should
        then be able to start working on a project of their own
       right away.

Caveat to convey throughout: this is my personal take in a very
       rapidly changing field. The workflow and approach are
       what work for me, and I'm happy to discuss them with
       anyone at any time.

## Background material
- `background.md`: my collection of ideas and thoughts I'd like
       to convey. They are not yet well structured.
- `recordings/`: the raw voice transcripts behind parts of `
       background.md`. Use them for reference only; `background.
       md` is the cleaned-up version.
```

```
## Tasks and stages
1. **Structure:** read `background.md` and propose how to bring
       the content into a proper structure: categories, sections
        and sub-bullets, in a sensible order. Do not write the
       presentation yet.
2. **Deliberate:** we walk through the proposed structure
       together and agree on the basic layout and on what should
        be conveyed. Where useful, suggest:
   - additional details or points I may have missed
   - concrete examples, e.g. short code snippets, an example `
       instructions.md`, or an example file/directory structure
       for a project folder
   - content that could be dropped or merged because it is not
       essential for a quick start
3. **Presentation:** once the structure is agreed, produce the
       slides (see below).

Before starting a stage, plan the subtasks and ask me about
       anything unclear up front. Anything non-critical can be
       left as a TODO note for later.
```

Excerpt. The full file is on GitHub: github.com/simonmartinecon/jumpstart-ai/blob/main/instructions.md

Ways of working

Asynchronous

- Longer tasks run in the background: half an hour, a few hours, overnight. Run several agents on different projects; reconcile when they finish.
- Settle all decisions up front, or defer non-critical ones as TODOs. What to avoid: after an hour, the agent stops to ask something you could have settled at the start.

> *Plan the subtasks for this stage and ask me about anything unclear before you start. Leave non-critical points as TODOs.*

Sparring partner

- Test ideas before taking them to colleagues: does the argument hold up against what we wrote elsewhere, and against the literature?
- Often points out what you have overlooked, such as a contradiction with an earlier section.

> *I think we should reframe the discussion in Section 5 as follows: . . . Assess this critically against the rest of the paper and the literature in literature/.*

Working interactively on one part is fine too, but most tasks can run asynchronously.

Pitfalls

- **Made-up or outdated citations.** My rule in `instructions.md`:

> *NEVER cite (add to the .bib file) any document that is not in literature/.*

  The agent may suggest literature, but I look up and download the papers myself. This way I have the real, latest versions, and cite only what I chose.
- **Backups.** The agent writes into your files. Keep file history (Dropbox, OneDrive, . . .), and read permission prompts before approving them.
- **Confidential data.** Check data licences and your institution's rules before giving the agent access to confidential data.

Putting it together: the jump start

1. Create a folder for the project.
2. Put in the context files: literature, data, a writing sample, your draft or notes.
3. Write `instructions.md` (10–15 minutes).
4. Open the agent in that folder (e.g. VS Code with Claude Code).
5. Tell it to read the instructions, and take it from there.

First prompts

> 💬 *Read instructions.md. Plan the subtasks and ask me about anything unclear before you start.*

> 💬 *Read status.md and continue where we left off.*

> 💬 *Do a thorough grammar, style and consistency check of paper.tex.*

Get started now



1. Create a folder

one per project



2. Put in the context files

literature, data, notes, a writing sample



3. Write `instructions.md`

scope, output, stages, style



4. Open the agent and prompt:

Read the instructions.

Thanks and further reading

- **Anton Korinek:** big thanks! A great source on using AI; I have benefited a lot from his talks and from interacting with him. His series of JEL articles on AI for economic research (latest: "AI Agents for Economic Research", 2025): https://www.korinek.com/Home
- **Scott Cunningham:** Scott's Mixtape Substack, on causal inference and how AI agents change empirical research: https://causalinf.substack.com
- **Paul Goldsmith-Pinkham:** great blog, Substack and video series: https://paulgp.com/blog.html

Questions, or a workflow that works better for you? Happy to discuss: simon.martin@univie.ac.at